UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ Accredited University

Name:- Kumar Saurav Suman **UID:- 25MCI10320**

Branch:- MCA [AI&ML] **Section/Group:- 25MAM-2B**

Semester:- 3rd **Date of Performance:- 23/10/2026**

Subject Name:- Technical Training II **Subject Code: 25CAP-707**

Experiment-6

Aim:-

To design and develop a full-stack Job Portal application with backend and MongoDB database integration using appropriate web technologies.

Tasks to be Done:-

The systematic workflow for developing this application includes the following operations:

1. Design a responsive Job Portal user interface.
2. Develop the frontend using appropriate web technologies.
3. Develop the backend server and API routes.
4. Connect the backend with MongoDB.
5. Create and manage collections for job-related data and users.
6. Implement CRUD operations.
7. Connect the frontend with backend APIs.
8. Implement job searching and job browsing functionality.
9. Store and retrieve job portal data from MongoDB.
10. Test the frontend, backend, API, and database integration.
11. Debug errors and improve application performance.

Production Architecture & Flow:

React Frontend → Node.js + Express APIs → JWT Auth → MongoDB

User Lifecycle:

Registration → Login → JWT Generation → Dashboard Access → View Jobs → Apply for Jobs → Application Status Tracking.

Implementation:-

index.html (React Application Entry)

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>JobFinder | Find Your Dream Job</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Backend – Secure Registration & Login APIs (JWT Integration)

```
// User Registration API with Password Hashing
router.post("/register", async (req, res) => {
  const { name, email, password } = req.body;
  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await User.create({
    name,
    email,
    password: hashedPassword
  });
  res.status(201).json({ message: "User Registered" });
});

// Login API with JWT Token Generation
router.post("/login", async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });

  if (!user) {
    return res.status(401).json({ message: "Invalid User" });
  }

  const valid = await bcrypt.compare(password, user.password);

  if (!valid) {
    return res.status(401).json({ message: "Wrong Password" });
```

```
  }

  const token = jwt.sign(
    { id: user._id },
    process.env.JWT_SECRET,
    { expiresIn: "1d" }
  );

  res.json({ token });
});

// Middleware: Validate JWT Token
module.exports = (req, res, next) => {
  const token = req.header("Authorization")?.replace("Bearer ", "");

  if (!token) {
    return res.status(401).json({ message: "Unauthorized" });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (error) {
    return res.status(401).json({ message: "Invalid Token" });
  }
};
```

React Frontend – Protected Routes & Data Fetching

```
import { Navigate } from "react-router-dom";

// Protected Route Component
function ProtectedRoute({ children }) {
  const token = localStorage.getItem("token");
  return token ? children : <Navigate to="/login" />;
```

```
}

// Fetching Jobs from API
useEffect(() => {
  axios.get("http://localhost:5000/api/jobs")
    .then(res => {
      setJobs(res.data);
    })
    .catch(error => {
      setMessage("Something Went Wrong");
    });
}, []);

// Securely Applying for a Job
function applyJob(jobId) {
  const token = localStorage.getItem("token");

  axios.post(
    "http://localhost:5000/api/applications",
    { jobId },
    {
      headers: {
        Authorization: `Bearer ${token}`
      }
    }
  )
  .then(() => {
    alert("Application Submitted");
  });
}
```

src/App.jsx – Search, Filter and Main State

```
function App() {
  const [currentPage, setCurrentPage] = useState('dashboard');
  const [isLoggedIn, setIsLoggedIn] = useState(false);
```

```
const [searchTerm, setSearchTerm] = useState('');
const [jobTypeFilter, setJobTypeFilter] = useState('All');
const [applications, setApplications] = useState([]);

const filteredJobs = jobsData.filter((job) => {
  const term = searchTerm.toLowerCase();

  const matchesSearch =
    job.title.toLowerCase().includes(term) ||
    job.company.toLowerCase().includes(term) ||
    job.location.toLowerCase().includes(term);

  const matchesType =
    jobTypeFilter === 'All' || job.type === jobTypeFilter;

  return matchesSearch && matchesType;
});

const handleApply = (job) => {
  if (!isLoggedIn) {
    showToast('Please login before applying.', 'warning');
    setCurrentPage('login');
    return;
  }

  const alreadyApplied = applications.some(
    (app) => app.jobId === job.id
  );

  if (alreadyApplied) {
    showToast('You have already applied to this job.', 'warning');
    return;
  }
```

```
  const newApplication = {
    id: Date.now(),
    jobId: job.id,
    title: job.title,
    company: job.company,
    appliedDate: new Date().toLocaleDateString('en-IN'),
    status: 'Applied'
  };

  setApplications([newApplication, ...applications]);
  showToast(`Applied to ${job.title} at ${job.company}!`, 'success');
};

// Component Render Logic...
}
```

Testing:-

The application implements extensive edge-case handling, including duplicate emails, invalid tokens, and empty fields.

Test Case	Action / Scenario	Expected Result
1	Register User	User Created (201)
2	Duplicate Email	Error Message: Email Already Exists (400)
3	Login User	JWT Generated & Authenticated (200)
4	Invalid Password	Error Message: Wrong Password (401)
5	Access Dashboard Without Login	Redirect to Login / Unauthorized (401)
6	Apply Job	Application Saved in DB (201)
7	Logout	Local Storage Cleared, Session Ended

Version Control & Deployment:-

```
# 1. Version Control Setup
git init
git add .
git commit -m "Initial Job Portal Project"
```

git remote add origin https://github.com/username/jobportal.git

git push -u origin main

2. Render Backend Environment Variables

MONGO_URI=mongodb+srv://<cluster-url>

JWT_SECRET=super_secure_secret

PORT=5000

OUTPUT:-

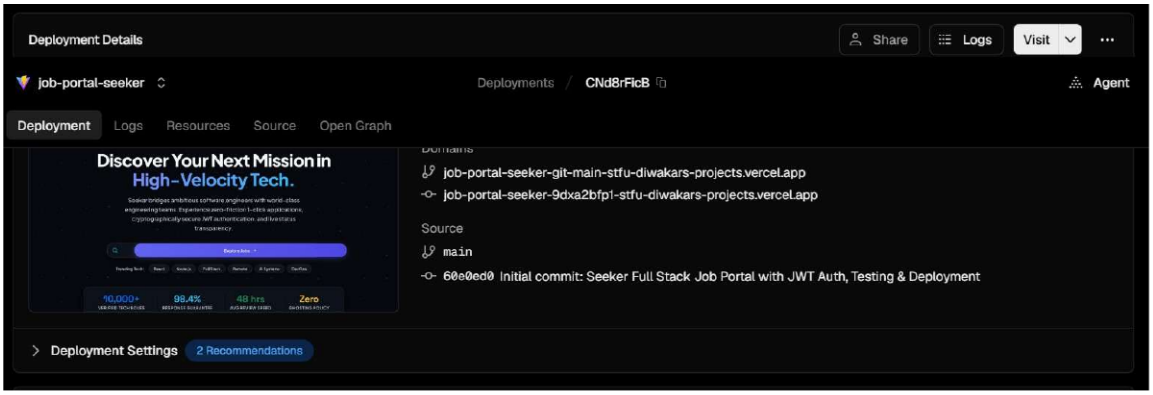


Figure 1: Frontend Job Portal Application running on Vercel.

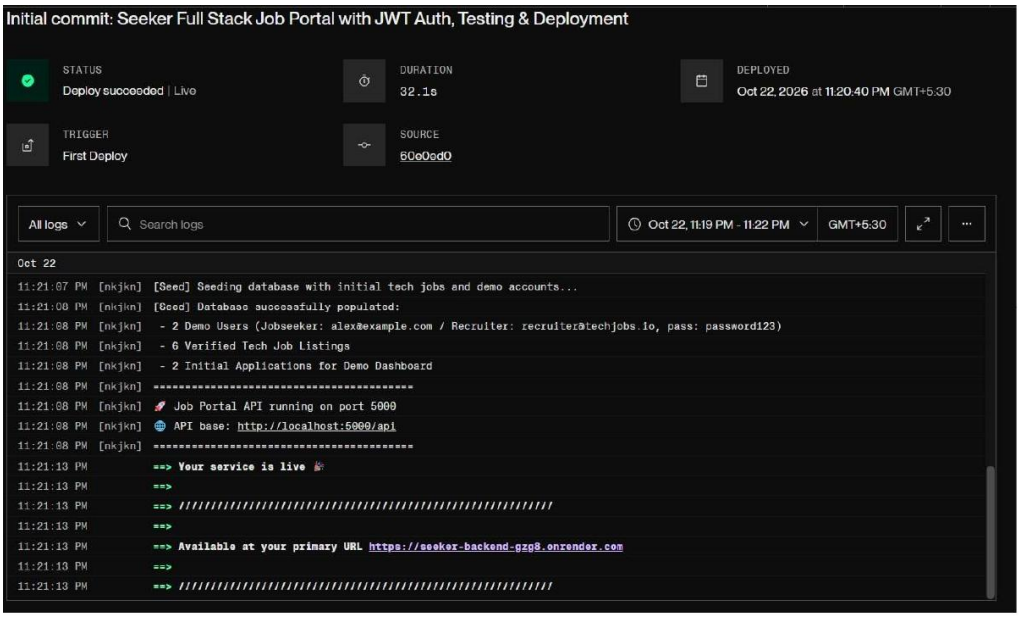


Figure 2: Backend verification showing API responses, JWT authentication, and database integration.

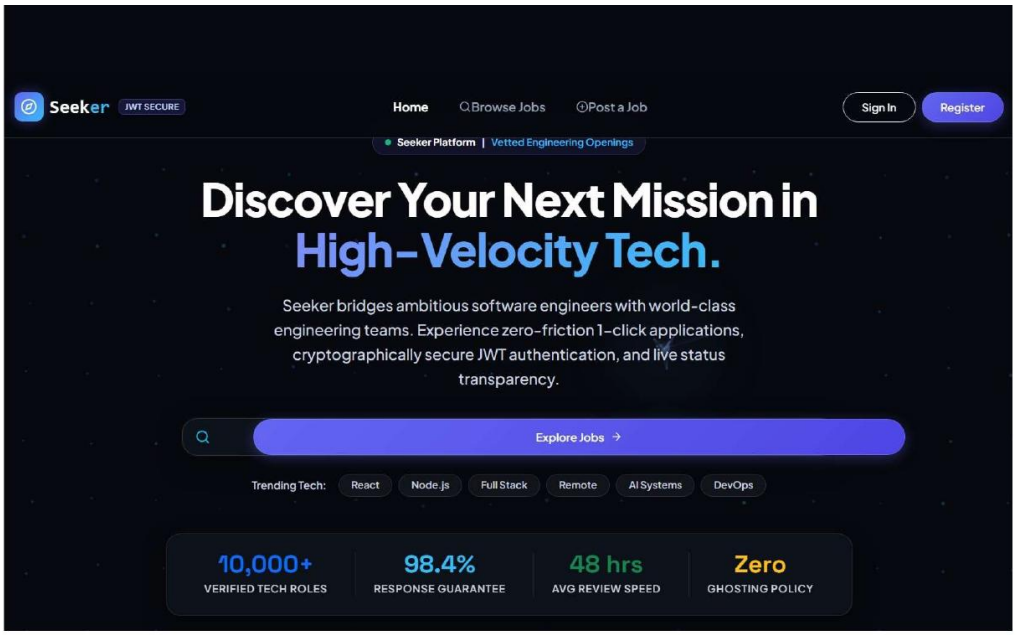


Figure 3: Frontend Job Portal Application displaying the dashboard, job listings, search functionality, and apply buttons.

Learning Outcome:-

- Architected a full-stack platform utilizing React.js, Node.js, Express, and MongoDB.
- Implemented stateless authentication utilizing JSON Web Tokens (JWT) and Bcrypt password hashing.
- Managed centralized API communication using Axios and handled edge cases across the stack.
- Executed end-to-end testing and continuous deployment workflows via GitHub, Vercel (Frontend), and Render (Backend).